

AULA 2

Laboratório: Primeira API REST



OBJETIVOS DA AULA



AO FINAL DESSA AULA, VOCÊ VAI SER CAPAZ DE...

- Criar um Controller do zero, com rotas GET e POST
- Criar um Service para concentrar a regra de negócio
- Implementar endpoints REST seguindo boas práticas (verbos e status codes corretos)
- Testar cada endpoint criado, direto no navegador ou via terminal



CRIAÇÃO DE CONTROLLERS



DECORATORS DE ROTA

```
@Controller('users')
export class UsersController {
  @Get()
  findAll() {}

  @Get('/:id')
  findOne(@Param('id') id: string) {}

  @Post()
  create(@Body() dto: any) {}
}
```

@Controller('users') define o prefixo — todas as rotas abaixo caem em /users.

DECORATORS DE PARÂMETRO

- `@Param('id')` — extrai um parâmetro da URL (ex: /users/3)
- `@Body()` — extrai o corpo da requisição (POST/PUT/PATCH)
- `@Query('nome')` — extrai a query string (?nome=valor)
- O controller só orquestra entrada e saída — delega tudo pro service



CRIAÇÃO DE SERVICES



ONDE MORA A REGRA DE NEGÓCIO

```
@Injectable()
export class UsersService {
  private users = [];
  private idCounter = 1;

  create(data) {
    const user = { id: this.idCounter++,
...data };
    this.users.push(user);
    return user;
  }

  findAll() {
    return this.users;
  }
}
```

POR QUE COMEÇAR EM MEMÓRIA

- Um array em memória simula o "banco" por enquanto — sem distração de infra
- O Controller nunca muda quando trocarmos isso por Prisma/Postgres na Aula 5/6
- @Injectable() marca a classe como gerenciável pelo container de DI do Nest
- O Controller recebe o service injetado no construtor — nunca faz "new UsersService()"



IMPLEMENTAÇÃO DE ENDPOINTS REST



OS ENDPOINTS DESTA AULA

```
GET  /users      → lista todos os usuários
GET  /users/:id   → retorna um usuário específico
POST /users      → cria um novo usuário
```

BOAS PRÁTICAS DESDE JÁ

- GET nunca deve alterar estado — só ler
- POST retorna 201 Created por padrão (o Nest já faz isso sozinho)
- Retorne o recurso criado/atualizado no corpo da resposta

